

UNIVERSITY OF THE ARMED FORCES - ESPE

Department of Exact Sciences

Object - Oriented Programming

NRC: 36863

UNIT 1 – HOMEWORK 5

GITHUB EDUCATION APPLICATION

Student Benefits Verification Using Institutional Email

JORGE EDISON LASCANO

October 14, 2026

ESPE 2609 – OBJECT-ORIENTED PROGRAMMING SW 36863

Instructor: Edison Lascano
Student: Dylan Cumbajin
Homework: 05
Unit: 1
Date: 15/11/ 2026

OBJECT ORIENTATION - GOOD PRACTICES

1.1 Conceptual Modeling

Conceptual modeling is the process of representing and organizing the ideas and most important elements of a system before building it.



Objetive

Its objetive is to help us:

- Understand the problem
- Analyze its main elements
- Desing a possible solution
- Communicate our ideas before implementing the system.

Importance

Purpose

- Understand → How the real business works.
- Visualize → How the current system works
- Envision → Imagine how we want the system to work in the future
- Document Desicions → Record important decesions
- Desing → Desing a possible solution
- Plan → Create a guide for software development.
- Communicate → Explain the project.

2 When do we use Conceptual Modeling?

From the beginning and at any point during the software development process.

1.3 Conceptual Modeling with UML

Visually represents a system from different perspectives → facilitating its
- comprehension
- desing
- development

Use Case Diagram

User functionalities → Who uses the system and what functions they can perform to archieve their goals.

Class Diagram

System structure → It represents the classes, their attributes, methods, and the relationships between them.

Interaction Diagram

Communication between objects → They allow objects to communicate with one another through messages.

State Chart

Changes in an object's state → It represents the differents states that an object can go through.

1.4 Organize Software into Loosely Coupled Layers

It is the process of dividing software into different layers, where each layer has a specific responsability

Main Software Layers:
- User Interface
- Application Logic
- Persistence Layers
- Communications
- Operating System

What is Loosely Coupled ? → Software component have minimal dependencies on each other

Main Advantage
- Easier maintenance
- Greater flexibility
- Easier modification
- Better software organization

Prefer Aggregation Over Inheritance

There are two ways to reuse elements:

Aggregation

One object contains or uses another object.

Inheritance

A class inherits attributes and methods from another class.

Why Prefer Aggregation? Because it facilitates changes and code reuse.

Program to an Interface or Abstraction

Using common methods instead of depending on specific classes

Programming with interface makes software more flexible organized and easy to mantain



Testing with Executable Unit Test Cases

Unit testing helps detect error, improve code quality and ensure that software works correctly

Why Is it Important?

- Detects error in the code and desing.
- Verifies that methods and classes work correctly
- Documents how the code should be used

INTRODUCTION to UML

WHAT IS UML?

Visual modeling language used to:

- represent
- desing
- understand
- software system

UML Diagrams

- Structural Diagrams: They represent the organization and structure of a system.
- Behavioral Diagrams: They represent how the system interact and how their states changes.

ULD BUILDING BLOCKS

Visual modeling language used to:

- represent
- classes
- objects
- relationships
- states
- messages

to describe a system

These elements are combined into diagrams that represent different aspect of the software



MODELING PERSPECTIVE

Software modeling has three main perspective.

- **Analysis**: Understand the problem and real world objects.
- **Specification**: Define how the system should work.
- **Implementation**: Represent the software components that will be developed.

SOFTWARE DEVELOPMENT ACTIVITIES

Software development includes analyzing:

- requirements
- designing
- implementing
- deploying
- a system

During these stages modeling activities are carried out:

Conceptual modeling, programming, testing and documentation

UML VIEWS AND ABSTRACTION

UML uses different view to describe a system allowing developers to focus on essential information without including unnecessary details.

GOOD UML MODELING PRACTICES

They must be
- clear
- organized
- easy to understand

use-cases

ACTORS

Actors are external entities that interact with a system to archive specific goals They be user, other software system, or hardware devices.

USE CASE DIAGRAMS

Visually represent the system's functionalities

elements
1. actors
2. use cases
3. relationships
4. system boundaries

USE CASES

They describe what user need to accomplish with a system

focusing on their goals rather than technical details

USE CASE RELATIONSHIPS

Use cases can be connected through three main relationship

- Extend: Add optional or conditional behavior to another use case
- Include: Represent required behavior reused by another use case
- Generalization: Represents a specialized version of a general use case

USE CASE VS SYSTEM INTERACTION

They explain what user want to achieve and why while the system describe how they interact with it to reach.

BENEFITS of USE CASES

They help identify requirements, organize functionalities and conduct testing.

OBJECTS-CLASSES AND BASIC CLASS DIAGRAMS

OBJECT: An element with its own identity state, and behavior

CLASS: A model that groups objects with similar characteristics and behaviors.

MODELIN PERSPECTIVE

- Analysis: Understand real-world objects and the problem.
- Specification: Define the logical structure and responsibilities of the System.
- Implementation: Describ how classes are developed in software

CLASS STRUCTURE

- Class Name: Identifies the class
- Attributes: Describ its properties or stored information
- Operation: Define its methods and behavior.

ATTRIBUTES AND OPERATIONS.

UML user visibility simbols + for public - for private and for protected

CLASS RELATIONSHIPS.

Association Connect objets from different

Generalization Represent inheritance or specialization

Agregation: Represent a whole part relationship

Dependency: Indicates that one class uses another and may be.